



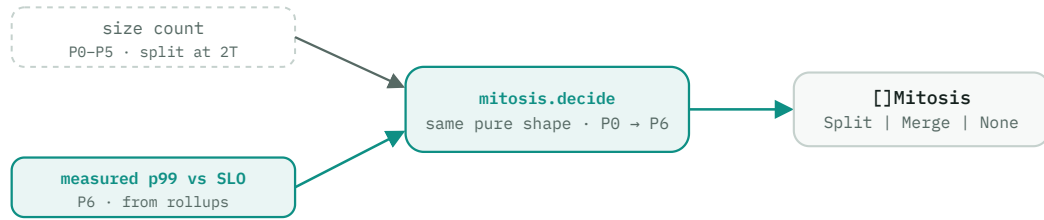
— COMPONENT DESIGN • PHASE 6

Self-tuning, and the loop closes.

P6 writes the last cores — mitosis that splits on a **measured signal** (the brief §03 upgrade), capability/locality placement refinement (short of a solver), and a matured reputation model — then validates a sustained run at a million nodes. Self-tuning turns out to be the same pure decision with a better input. From two machines to a million, the FCIS boundary never moved.

01 Self-tuning is the same core, better input

Back in P0, mitosis split a cell when its *size* crossed 2T — a count as a proxy for "coordination is getting slow." P6 delivers the graduation the brief §03 promised: split when the **measured signal** (barrier-completion p99, queue latency) actually crosses the SLO. The elegant part — and the whole thesis of this series in one move — is that `decide` keeps its exact shape. The trigger gets a richer *input*, fed by P4's observability rollups; the pure function and its `[]Mitosis` output are unchanged.



a count is just one signal – signal-based subsumes count-based

The differentiator, self-tuned, with no new machinery. Because a count is just one kind of signal, the signal-based trigger subsumes the count-based one — P0's mitosis tests still pass, and P6 only adds tests for the new signal path. The cell that divides now divides on real latency, not a proxy.

02 The last cores

Signal-based adaptive mitosis

MITOSIS · §03 UPGRADE

Responsibility — split on a measured signal crossing the SLO, not just a size count.

Same pure `decide` shape; the signal is an input the shell supplies from P4's rollups.

◆ FUNCTIONAL CORE · PURE

```
func decide(cells []CellSignal, cfg SignalThresholds,
            cooldowns map[CellID]Instant, now Instant) []Mitosis
    // CellSignal = { size int; p99 Duration; tput float }
    // split when p99 > slo(coupling); merge when well under
func signalThreshold(c Coupling, slo SLO) Threshold
```

▮ IMPERATIVE SHELL · I/O

- feed the measured signal from observability rollups (P4)
- execute `Split` / `Merge` exactly as before (unchanged since P0/P2)

Boundary — cell signals + thresholds + clock in; `[]Mitosis` out. Signal-based subsumes count-based, so P0's mitosis tests still pass and only the signal path is new — the differentiator, self-tuned, with no new machinery.

Capability/locality placement refinement

PLACEMENT · DELTA

Responsibility — prefer machines by capability match and network locality (rack / AZ / region distance). A bounded, single-pass ranking — explicitly not a solver (brief §18).

◆ FUNCTIONAL CORE · PURE

```
func rank(t Task, cands []CellView,
         loc LocalityGraph) []Ranked
    // greedy best-fit: capability match + locality distance
func bestFit(t Task, cands []CellView,
            loc LocalityGraph) Placement
```

▮ IMPERATIVE SHELL · I/O

- gather candidate snapshots + the locality graph
- dispatch to the top-ranked cell

Boundary — task + candidates + locality in; a ranked list / a placement out. A one-pass greedy pick that provably terminates — no iterative optimization, honoring §18's "no heavy solver."

Extends P0's `place` and P1's `placeAcross` without changing their callers.

Reputation maturity

VERIFICATION · DELTA

Responsibility — mature P3's reputation with time-decay (stale trust fades) and tiered thresholds (faster convergence for consistent contributors). Still pure

update / weight , now with decay and tiers.

◆ FUNCTIONAL CORE · PURE

```
func decay(rep Reputation, elapsed Duration) Reputation
    // stale reputation fades toward zero
func tier(rep Reputation) RepTier    // verification bucket
// P3's update / weight unchanged
```

▮ IMPERATIVE SHELL · I/O

- run a periodic decay pass
- persist the reputation store

Boundary — reputation + elapsed in; a decayed reputation / a tier out. Decay stops a once-good machine from coasting forever; the tests are P3's property style (monotonic on honest work, fades on absence).

03 The design realized

With the last cores written, the sustained run at ~1M nodes — self-tuning on, all six phases composed — is the final GA gate. And the map below is the whole point of having held the line: **every phase added pure decision cores and thin shells, and the boundary never moved.** This is the functional core of Swarm, entire:

PHASE	PURE DECISION CORES ADDED
P0	admit · place · decide(mitosis) · registration step · run step · template decompose/merge
P1	route · mergeGlobal · placeAcross · selectRegion · summarize
P2	driver step (barrier/leader/message) · resume · checkpoint due/snapshot/restore · mitosis gate · admitGang · deadline/isDead
P3	grants · verifyModule · assign · verdict · reputation update/weight · shouldProbe/check · admitOpen · verifySignature
P4	shardOf · rollup · admitUnderLoad · nextDrain · skewSafe
P5	withinQuota · nextFair · recoveryPlan · evaluate/shouldAlert · verifyAttestation
P6	signal-based decide · rank/bestFit · decay/tier

From two machines to a million

Every decision in Swarm — when a cell divides, where a task lands, whether a quorum agrees, how a barrier steps, when to page, what to recover — is a **pure function** you can read and test in isolation. Every effect — gossip, gRPC, WASM, FoundationDB, a GPU all-reduce — lives in a **thin shell** that gathers inputs and carries decisions out. The swarm scaled across six phases and five orders of magnitude, and the line between the two never had to move. That is the design, realized.